

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUADA – Práctica 3

Presentación

En la Práctica 2 se han implementado tres microservicios del caso **Photo&Film4You**. En esta Práctica 3 verificaremos el funcionamiento correcto de los microservicios mediante pruebas automatizadas.

Competencias

En esta Práctica se trabajan las siguientes competencias del Grado de Ingeniería Informática:

- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.
- Aplicación de las técnicas específicas de la Ingeniería del Software a las diferentes etapas del ciclo de vida de un proyecto.

Objetivos

Los objetivos de la Práctica 3 son:

- Reconocer y proponer una estrategia de pruebas para una arquitectura basada en microservicios
- Reconocer y utilizar la simulación de dependencias de clases en las pruebas unitarias
- Reconocer y utilizar el uso de pruebas para verificar el contrato entre microservicios
- Reconocer y diferenciar la cobertura de la calidad de las pruebas, usando pruebas de mutación

Descripción de la Práctica 3 a realizar

La Práctica 3 se compone de 4 ejercicios. En cada uno de ellos hay que **justificar** todas las decisiones.

Para todos los ejercicios debéis partir de la solución de la Práctica 2 publicada en: <https://github.com/orgs/UOC-SPRING-2026/repositories>.

Ejercicio 1 - Estrategia de pruebas (2 puntos)

El archivo `CourseApplicationTests.java` solo contiene una prueba que verifica que el contexto de Spring arranca. Teniendo en cuenta la arquitectura de microservicios de la aplicación:

- Define una estrategia de pruebas para esta aplicación, indicando qué niveles de prueba existen, qué tipos de pruebas son necesarios para cada nivel y qué aspectos concretos cubrirían (1,5 puntos)

Justifica en detalle las decisiones y aporta adicionalmente un diagrama resumen en la que se indique para cada nivel de pruebas qué tipos de pruebas se incluyen y qué librerías o frameworks de pruebas automáticas se proponen.

- Resume qué pruebas aplicarías al microservicio de Cursos en cada nivel y tipo definido en la estrategia (0,5 puntos).

Solución

Existen diferentes alternativas de estrategias. Lo importante es su justificación.

Una posible distribución de niveles de prueba es la siguiente:

- Pruebas unitarias
- Pruebas de contrato
- Pruebas de integración
- Pruebas de componente
- Pruebas E2E

Pruebas Unitarias (Unit Tests)

Probamos la lógica de negocio aisladamente de toda la infraestructura, usando Junit, Mockito y AssertJ. Mockeamos (simulamos) las dependencias. Son las pruebas más rápidas, las más baratas de mantener y cubren la mayor densidad de lógica de negocio.

Tipo	Qué verificamos
Lógica de dominio	Transiciones de estado válidas e inválidas del Course, reglas de inscripciones (no duplicados, curso en estado correcto), validación de instructor
Service layer	CourseServiceImpl y EnrollmentServiceImpl con repositorios mockeados: orquestación, excepciones, flujos happy-path y error
Mapeo domain-entity	Se preservan todos los campos correctamente en los mapeos CourseEntity.toDomain() / fromDomain()

Pruebas de Contrato (Contract Tests)

Las pruebas de contrato pueden considerarse parte pruebas unitarias (lado del consumidor) y parte pruebas de integración (lado del proveedor). Si habéis ubicado las pruebas de contrato después de las pruebas de integración se considera también correcto.

Las pruebas de contrato del lado consumidor las hemos clasificado como pruebas unitarias porque Pact genera un servidor mock local, no se requiere infraestructura ni red, y se ejecutan en milisegundos. Aunque conceptualmente validan un acuerdo entre servicios, su mecanismo de ejecución es idéntico al de cualquier prueba unitaria. Podemos usar Pact JVM Consumer (pact-jvm-consumer-junit5) y Pact Broker (para publicar los contratos).

En cambio, la verificación de contratos del lado proveedor la podemos clasificar aquí porque puede requerir levantar el servicio completo con Spring, aunque también puede ser una prueba unitaria.

Tipo	Qué verificamos
Pruebas de contrato del lado del consumidor	Pruebas de Pact del lado consumidor: definen qué espera el Course Service de los servicios externos (User Service, Microcredential Service). Las ejecutamos contra un mock local generado por Pact, sin levantar ningún servicio
Pruebas de contrato del lado del productor	Verificación del lado proveedor con Pact: levanta el servicio real y verifica que cumple los contratos publicados por los consumidores.

Pruebas de Integración (Integration Tests)

Probamos interacciones reales con la infraestructura, desde el código, usando Spring Boot Testing, TestContainers y Wiremock (para dependencias externas que no controlamos).

La capa de infraestructura contiene conversiones entity-domain, queries JPA y parsing de respuestas HTTP. Con Testcontainers podemos proporcionar un PostgreSQL idéntico al de producción sin depender de infraestructura compartida. WireMock nos permite simular servicios externos de forma determinista, incluyendo errores de red y respuestas lentas.

Tipo	Qué verificamos
Repository (JPA)	Que SpringDataCourseRepository y SpringDataEnrollmentRepository persisten y recuperan datos correctamente contra un PostgreSQL real mediante Testcontainers
REST Controller	Que CourseRestController serializa/deserializa JSON, responde con estados HTTP correctos y aplica las validaciones @Valid de los request DTOs
REST Client	Que UserRepositoryImpl traduce correctamente las respuestas del User Service al contrato del dominio (UserRepository), configuración del RestTemplate, interpolación de URL, deserialización de GetUserResponse y traducción de escenarios HTTP (éxito, error del servidor) al comportamiento esperado por la capa de dominio

Pruebas de Componente (Component Tests)

Probamos un **microservicio completo desplegado** como una unidad, solo las dependencias externas son simuladas. Es el nivel más alto de prueba que puede ejecutarse sin depender de otros servicios reales. Usamos los frameworks: SpringBootTest, Testcontainers (PostgreSQL), WireMock y REST Assured (o TestRestTemplate).

Verificamos que todas las capas (controller, service, repository, base de datos) funcionan correctamente dentro del servicio. Estas pruebas detectan errores de wiring de Spring, transacciones, configuración y serialización request-response. Son más lentas que las unitarias o de integración, pero más rápidas y deterministas que las E2E.

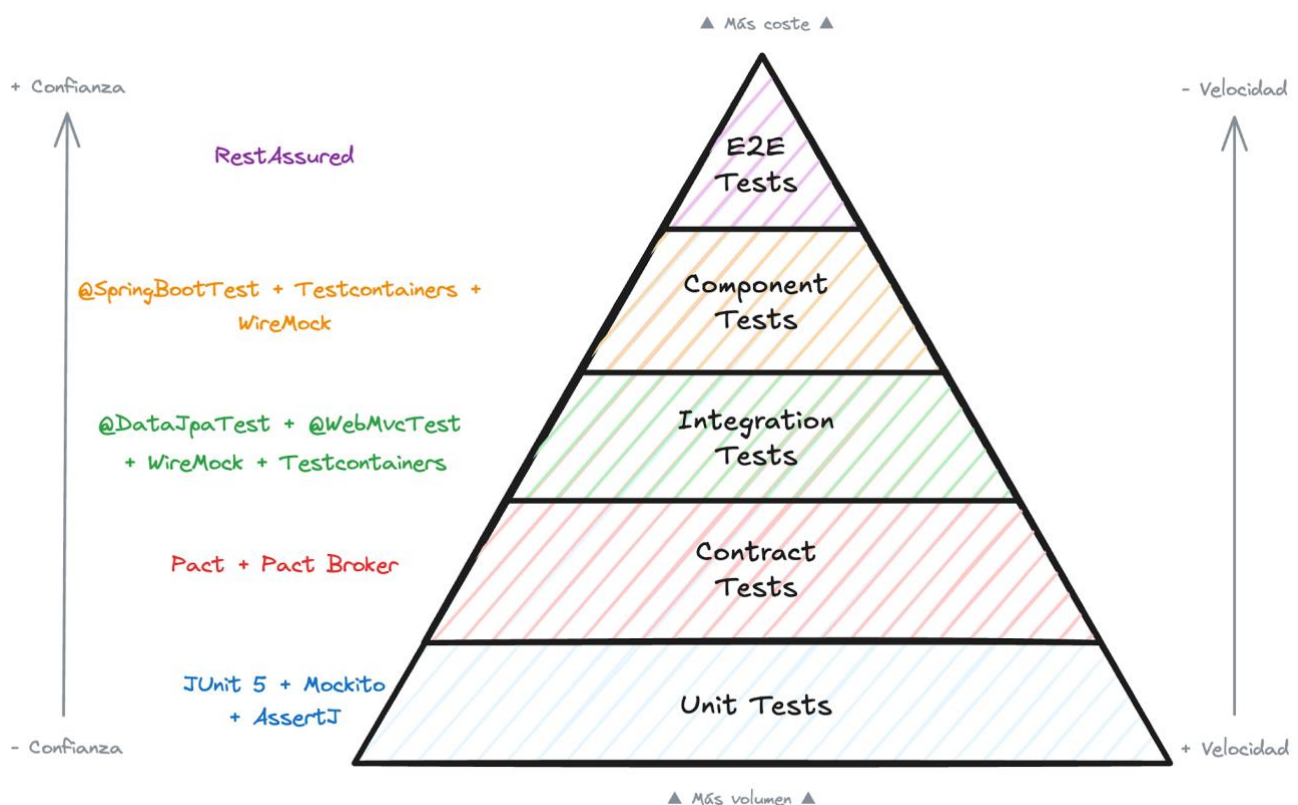
Tipo	Qué verificamos
Full-stack aislado	Course Service levantado con @SpringBootTest + Testcontainers (PostgreSQL) + WireMock (User Service y Microcredential Service). Se prueban flujos end-to-end dentro del servicio
Máquina de estados completa	Ciclo de vida completo del curso: DRAFT → ENROLLMENT_OPEN → ACTIVE → PENDING_CLOSUE → CLOSED pasando por todos los endpoints REST
Flujos de matrícula	Crear curso, abrir enrollment, matricular múltiples estudiantes, verificar lista, cerrar

Pruebas E2E

En las pruebas E2E (End-to-End) probamos múltiples servicios reales interactuando en un entorno similar al de producción. Son las pruebas que dan mayor confianza, pero también las más costosas. Son lentas, frágiles y requieren toda la infraestructura. Por eso las limitamos a los “happy paths” críticos del negocio. Podemos detectar problemas de configuración de red, incompatibilidades de versiones entre servicios y fallos en la cadena completa de eventos. Podemos usar los frameworks:

- Docker Compose (arrancamos toda la infraestructura real: PostgreSQL + Kafka + todos los servicios)
- REST Assured

Tipo	Qué verificamos
Flujo cross-servicios	Crear curso (Course Service) → matricular usuario validado (User Service real) → cerrar curso → generar microcredencial (Microcredential Service real) → notificación (Notification Service vía Kafka)



Ejercicio 2 - Pruebas unitarias y reporting (2 puntos)

Apartado 1 (1 punto)

Escribe una prueba **unitaria** para `CourseServiceImpl.createCourse()` usando JUnit5, Mockito y AssertJ. La prueba debe verificar los siguientes escenarios:

- El instructor existe → el curso se crea con estado DRAFT
- El instructor no existe → se lanza `UserNotFoundException` y no se invoca el método `createCourse` del `CourseRepository`

Justifica qué dependencias necesitas mockear y por qué no puedes instanciar `CourseServiceImpl` directamente sin mocks.

Notas:

En las pruebas unitarias no hacemos uso de ninguna infraestructura ni dependencia real. No usamos comunicación HTTP, servidores, bases de datos, servicios externos u otros microservicios. Todo aspecto relacionado con dependencias debe simularse (con **Mockito**), excepto las clases de dominio (`Course`, `Enrollment`, ...).

Solución

```
@ExtendWith(MockitoExtension.class)
class CourseServiceImplTest {

    @Mock
    private CourseRepository courseRepository;

    @Mock
    private UserRepository userRepository;

    @Mock
    private EnrollmentRepository enrollmentRepository;

    @Mock
    private EnrollmentService enrollmentService;

    @InjectMocks
    private CourseServiceImpl courseService;

    @Test
    @DisplayName("createCourse - instructor exists → course created with DRAFT status")
    void createCourse_instructorExists_returnsCourseIdWithDraftStatus() {
        String instructorEmail = "instructor@uoc.edu";
        Course course = Course.builder()
```

```

        .instructor(instructorEmail)
        .title("Diseño de Software")
        .build();

when(userRepository.findInstructorByEmail(instructorEmail)).thenReturn(true);
when(courseRepository.createCourse(any(Course.class))).thenReturn(1L);

Long courseId = courseService.createCourse(course);

assertSoftly(softly -> {
    softly.assertThat(courseId).isEqualTo(1L);
    softly.assertThat(course.getStatus()).isEqualTo(CourseStatus.DRAFT);
    softly.check(() -> verify(userRepository).findInstructorByEmail(instructorEmail));
    softly.check(() -> verify(courseRepository).createCourse(course));
});
}

@Test
@DisplayName("createCourse - instructor does not exist → throws UserNotFoundException, courseRepository never called")
void createCourse_instructorNotFound_throwsAndNeverCreatesCourse() {
    String instructorEmail = "unknown@uoc.edu";
    Course course = Course.builder()
        .instructor(instructorEmail)
        .title("Diseño de Software")
        .build();

    when(userRepository.findInstructorByEmail(instructorEmail)).thenReturn(false);

    assertThatThrownBy(() -> courseService.createCourse(course))
        .isInstanceOf(UserNotFoundException.class)
        .hasMessageContaining(instructorEmail);

    verify(userRepository).findInstructorByEmail(instructorEmail);
    verify(courseRepository, never()).createCourse(any());
}
}

```

No podemos instanciar CourseServiceImpl sin mocks porque tiene 4 dependencias reales que finalmente de forma transitiva dependen de la infraestructura.

- Dependencias inyectadas por constructor. @RequiredArgsConstructor genera un constructor con CourseRepository, UserRepository, EnrollmentRepository y EnrollmentService. Todos son interfaces sin implementación en memoria — sus implementaciones reales hablan con PostgreSQL y servicios HTTP externos.
- UserRepository. Se mockea porque findInstructorByEmail hace una llamada HTTP al microservicio de usuarios. En una prueba unitaria no queremos levantar infraestructura externa.

- CourseRepository. Se mockea porque su implementación real (CourseRepositoryImpl) usa Spring Data JPA contra PostgreSQL. Necesitamos controlar qué devuelve createCourse sin tocar la base de datos.
- EnrollmentRepository y EnrollmentService. Se declaran como @Mock porque Mockito con @InjectMocks necesita **todas** las dependencias del constructor satisfechas, **aunque createCourse no las use directamente**.

Apartado 2 (1 punto)

Utilizando como base las pruebas realizadas en el apartado anterior, debes configurar el adaptador de Allure correspondiente a Junit5 (<https://allurereport.org/docs/junit5/>) para transformar los logs de consola en un informe visual en HTML.

En el informe de entrega PDF se debe incluir:

- Pasos realizados para integrar el proyecto con Allure, junto con fragmentos relevantes de los archivos de configuración y del código de pruebas si se han modificado.
- Descripción breve del proceso para generar el archivo HTML de resultados y de su contenido (Overview, Timeline), apoyada por capturas de pantalla como evidencia.

Entrega adicionalmente un ZIP que incluya los archivos HTML y ficheros relacionados generados con Allure.

Solución

Fichero pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <properties>
    <allure.version>2.25.0</allure.version>
    <aspectj.version>1.9.21</aspectj.version>
  </properties>
  <dependencies>

    <dependency>
      <groupId>io.qameta.allure</groupId>
      <artifactId>allure-junit5</artifactId>
      <version>${allure.version}</version>
      <scope>test</scope>
```



```

</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <configuration>
        <argLine>
          -javaagent:"${settings.localRepository}/org/aspectj/aspectjweaver/${aspectj.version}/aspectjweaver-
${aspectj.version}.jar"
        </argLine>
      </configuration>
      <dependencies>
        <dependency>
          <groupId>org.aspectj</groupId>
          <artifactId>aspectjweaver</artifactId>
          <version>${aspectj.version}</version>
        </dependency>
      </dependencies>
    </plugin>
  </plugins>
</build>

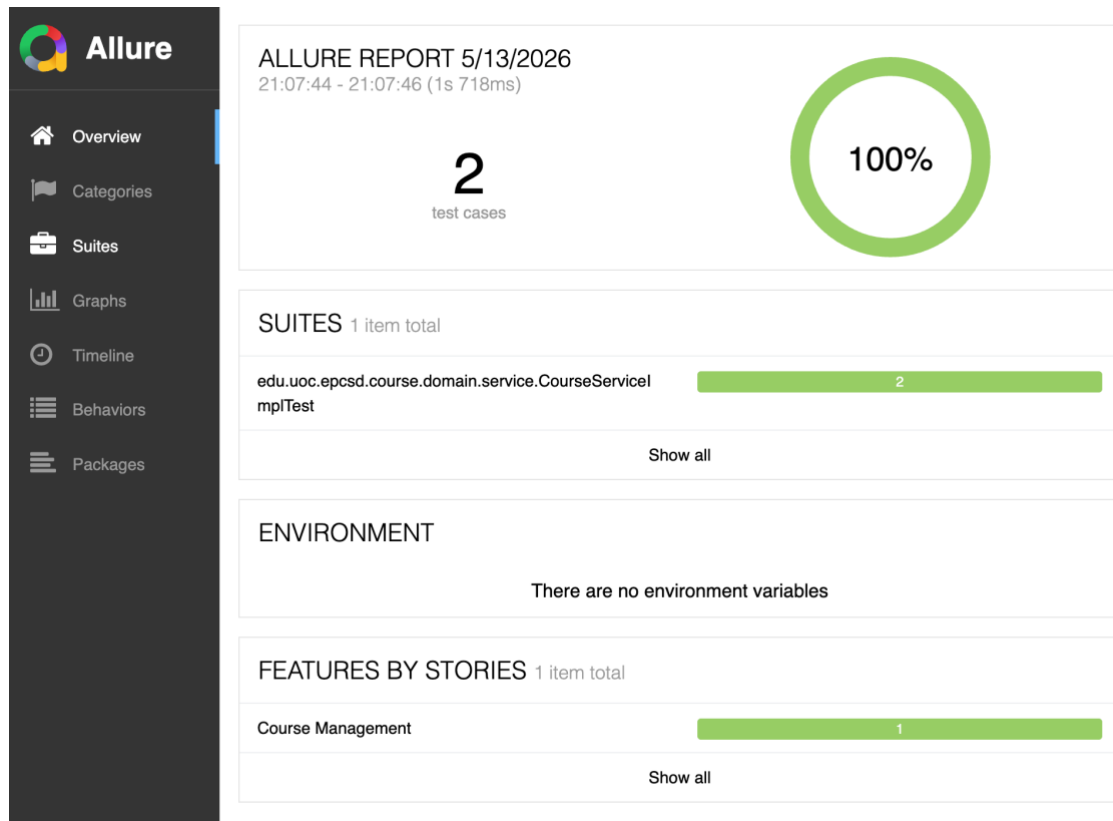
</project>

```

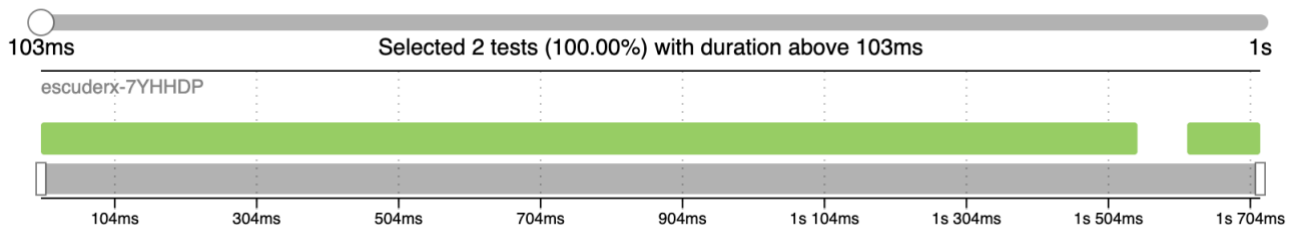
Necesitamos AspectJ weaver como javaagent para que Allure intercepte la ejecución de las pruebas y genere resultados.

Generamos los resultados desde el terminal con:

```
$ allure:serve
```



El Overview muestra que ambos casos de prueba se completaron con éxito.



En el Timeline vemos como la duración del primer caso de prueba ha sido mucho mayor que la del segundo. El motivo es que la primera prueba carga toda la infraestructura de Mockito, que la segunda prueba ya reutiliza.

Ejercicio 3 – Pruebas de contrato (3,5 puntos)

Agregar pruebas de contratos es clave para garantizar que los servicios y microservicios funcionen correctamente juntos.

El objetivo de este ejercicio es generar pruebas automáticas de contrato utilizando **Pact** para validar los contratos establecidos entre *CourseService* (como consumidor) y *MicroCredentialsService* (como productor).

- a) Escenario 1. Creación exitosa de microcredenciales (/microcredentials/1/create): 201 Created o 200 OK
- b) Escenario 2. Curso no encontrado al crear unas microcredenciales (/microcredentials/999/create): 402

Debéis realizar:

- La implementación de la prueba de Pact en el lado del consumidor donde se definen las interacciones para los dos escenarios solicitados (debe incluirse la configuración del PactBuilder y el uso del servidor mock).
- Generar con Pact el archivo JSON que contiene las expectativas del contrato
- La implementación de la prueba de Pact en el lado del proveedor.
- Informe de ejecución con capturas o logs:
 - o Lado Consumidor: Evidencia de que las pruebas pasaron y el archivo JSON se generó correctamente.
 - o Lado Proveedor: Evidencia de que el proveedor validó con éxito el contrato enviado por el consumidor.

Notas:

Podéis usar como referencia para ayudaros a desarrollar las pruebas:

- <https://www.baeldung.com/pact-junit-consumer-driven-contracts>

Solución

Consumidor (CourseService)

```
@ExtendWith(PactConsumerTestExt.class)
class MicroCredentialsServiceContractTest {

    private final RestTemplate restTemplate = new RestTemplate();

    private Map<String, String> jsonHeaders() {
        return Map.of("Content-Type", "application/json");
    }

    @Pact(consumer = "CourseService", provider = "MicroCredentialsService")
    V4Pact createMicrocredentialsSuccess(PactDslWithProvider builder) {
        return builder
            .given("course 1 exists")
            .uponReceiving("a POST request to create microcredentials for existing course 1")
            .method("POST")
            .path("/microcredentials/1/create")
            .willRespondWith()
            .status(201)
            .headers(jsonHeaders())
            .body("true")
            .toPact(V4Pact.class);
    }

    @Pact(consumer = "CourseService", provider = "MicroCredentialsService")
    V4Pact createMicrocredentialsCourseNotFound(PactDslWithProvider builder) {
        return builder
            .given("course 999 does not exist")
            .uponReceiving("a POST request to create microcredentials for non-existing course 999")
            .method("POST")
            .path("/microcredentials/999/create")
            .willRespondWith()
            .status(400)
            .toPact(V4Pact.class);
    }

    @Test
    @Story("Successful microcredential creation")
    @Severity(SeverityLevel.CRITICAL)
    @PactTestFor(pactMethod = "createMicrocredentialsSuccess")
    void createMicrocredentials_courseExists_returns201(MockServer mockServer) {
        String url = mockServer.getUrl() + "/microcredentials/1/create";

        ResponseEntity<Boolean> response = restTemplate.postForEntity(url, null, Boolean.class);

        assertSoftly(softly -> {
```

```

        softly.assertThat(response.getStatusCodeValue()).isEqualTo(201);
        softly.assertThat(response.getBody()).isTrue();
    });
}

@Test
@Story("Course not found")
@Severity(SeverityLevel.CRITICAL)
@PactTestFor(pactMethod = "createMicrocredentialsCourseNotFound")
void createMicrocredentials_courseNotFound_returns402(MockServer mockServer) {
    String url = mockServer.getUrl() + "/microcredentials/999/create";

    HttpClientErrorException ex = catchThrowableOfType(
        () -> restTemplate.postForEntity(url, null, Boolean.class),
        HttpClientErrorException.class);

    assertSoftly(softly -> {
        softly.assertThat(ex).isNotNull();
        softly.assertThat(ex.getRawStatusCode()).isEqualTo(402);
    });
}
}

```

Contrato

```

{
  "consumer": {
    "name": "CourseService"
  },
  "interactions": [
    {
      "description": "a POST request to create microcredentials for existing course 1",
      "key": "418cf1d1",
      "pending": false,
      "providerStates": [
        {
          "name": "course 1 exists"
        }
      ],
      "request": {
        "method": "POST",
        "path": "/microcredentials/1/create"
      },
      "response": {
        "body": {
          "content": true,
          "contentType": "application/json",
          "encoded": false
        },
        "headers": {
          "Content-Type": [

```

```

    "application/json"
  ],
  "status": 201
},
"transport": "https",
"type": "Synchronous/HTTP"
},
{
  "description": "a request to create microcredentials for an existing course",
  "key": "c06b89cf",
  "pending": false,
  "providerStates": [
    {
      "name": "course 1 exists"
    }
  ],
  "request": {
    "method": "POST",
    "path": "/microcredentials/1/create"
  },
  "response": {
    "status": 201
  },
  "transport": "https",
  "type": "Synchronous/HTTP"
},
{
  "description": "a POST request to create microcredentials for non-existing course 999",
  "key": "ca87e72a",
  "pending": false,
  "providerStates": [
    {
      "name": "course 999 does not exist"
    }
  ],
  "request": {
    "method": "POST",
    "path": "/microcredentials/999/create"
  },
  "response": {
    "status": 402
  },
  "transport": "https",
  "type": "Synchronous/HTTP"
},
{
  "description": "a request to create microcredentials for a non-existing course",
  "key": "df5d5fcc",
  "pending": false,
  "providerStates": [
    {
      "name": "course 999 does not exist"
    }
  ],
  "request": {
    "method": "POST",

```

```

    "path": "/microcredentials/999/create"
  },
  "response": {
    "status": 402
  },
  "transport": "https",
  "type": "Synchronous/HTTP"
},
"metadata": {
  "pact-jvm": {
    "version": "4.6.5"
  },
  "pactSpecification": {
    "version": "4.0"
  }
},
"provider": {
  "name": "MicroCredentialsService"
}
}

```

Y lo copiamos en el directorio `src/test/resources` del proyecto del servicio de microcredenciales.

Productor (MicrocredentialsService):

```

@ExtendWith(SpringExtension.class)
@SpringBootTest(
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
    classes = MicrocredentialRestController.class
)
@EnableAutoConfiguration(exclude = {
    DataSourceAutoConfiguration.class,
    HibernateJpaAutoConfiguration.class,
    KafkaAutoConfiguration.class
})
@Provider("MicroCredentialsService")
@PactFolder("src/test/resources")
public class MicrocredentialProviderPactTest {

    @LocalServerPort
    private int port;

    @MockBean
    private MicrocredentialService microcredentialService;

    @BeforeEach
    void setUp(PactVerificationContext context) {
        context.setTarget(new HttpTestTarget("localhost", port));
    }
}

```



```

@TestTemplate
@ExtendWith(PactVerificationInvocationContextProvider.class)
void pactVerificationTestTemplate(PactVerificationContext context) {
    context.verifyInteraction();
}

@State("course 1 exists")
void courseOneExists() {
    Mockito.when(microcredentialService.requestCourseMicrocredentials(1L))
        .thenReturn(true);
}

@State("course 999 does not exist")
void courseNineNineNineDoesNotExist() {
    Mockito.when(microcredentialService.requestCourseMicrocredentials(999L))
        .thenThrow(new CourseNotFoundException(999L));
}
}

```

Ejecución del test del productor (microcredenciales):

```
mvn test -Dtest=MicrocredentialProviderPactTest
```

Fijarse como tenemos tantos resultados como interacciones se han definido (4).

Realizamos también la ejecución en el consumidor (curso):

```
mvn test -Dtest=MicroCredentialsServiceContractTest
```

Ejercicio 4 – Pruebas de mutación (2,5 puntos)

Se ha implementado una nueva clase *CourseValidator* que contiene lógica de negocio para validar diferentes aspectos en la creación de los cursos en el sistema. Actualmente, incluye las siguientes validaciones o criterios de aceptación:

- Validación de precio: Rango permitido entre 0 y 10.000€
- Validación de duración: Entre 1 y 500 horas
- Cálculo de descuentos: Basado en rangos de precio

El código es el siguiente:

```
package edu.uoc.epcsd.course.domain.validator;

public class CourseValidator {

    private static final int MIN_PRICE = 0;
    private static final int MAX_PRICE = 10000;
    private static final int MIN_DURATION = 1;
    private static final int MAX_DURATION = 500;

    /**
     * Validates whether a course price is valid.
     * It must be between 0 and 10000.
     */
    public boolean isValidPrice(Long price) {
        if (price == null) {
            return false;
        }
        return price >= MIN_PRICE && price <= MAX_PRICE;
    }

    /**
     * Validates whether the course duration is valid.
     * It must be between 1 and 500 hours.
     */
    public boolean isValidDuration(Long duration) {
        if (duration == null) {
```

```

        return false;
    }
    return duration >= MIN_DURATION && duration <= MAX_DURATION;
}

/**
 * Calculates the applicable discount based on the price.
 * - Price >= 5000: 20% discount
 * - Price >= 2000: 10% discount
 * - Otherwise: 5% discount
 */
public int calculateDiscount(Long price) {
    if (price == null || price < 0) {
        return 0;
    }
    if (price >= 5000) {
        return 20;
    } else if (price >= 2000) {
        return 10;
    } else {
        return 5;
    }
}
}

```

Así mismo se ha implementado una prueba unitaria que verifica estos criterios de aceptación:

```

package edu.uoc.epcsd.course.domain.validator;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

class CourseValidatorTest {
    private CourseValidator validator;

```

```

@BeforeEach
void setUp() {
    validator = new CourseValidator();
}

@Test
void testIsValidPrice() {
    assertThat(validator.isValidPrice(5000L)).isTrue();
    assertThat(validator.isValidPrice(null)).isFalse();
}

@Test
void testIsValidDuration() {
    assertThat(validator.isValidDuration(100L)).isTrue();
    assertThat(validator.isValidDuration(null)).isFalse();
}

@Test
void testCalculateDiscount() {
    assertThat(validator.calculateDiscount(7000L)).isEqualTo(20);
    assertThat(validator.calculateDiscount(3000L)).isEqualTo(10);
    assertThat(validator.calculateDiscount(1000L)).isEqualTo(5);
    assertThat(validator.calculateDiscount(null)).isEqualTo(0);
}
}

```

En este ejercicio debéis realizar los siguientes pasos:

1. Revisa los tests (*testIsValidPrice*, *testIsValidDuration*, *testCalculateDiscount*). ¿Qué porcentaje de cobertura de líneas tiene la clase *CourseValidator*? Describe el significado de este valor para esta clase.
2. Configura y ejecuta **piTest** (<https://pitest.org/>) en el proyecto. Explica cómo se configura piTest, qué archivos y fragmentos se modifican, y cómo generar el informe HTML de resultados.

3. Revisa el informe HTML de piTest y justifica las métricas siguientes asociadas a las clases *CourseValidatorTest* y *CourseValidator*:
 - a. Mutation coverage
 - b. Line Coverage
 - c. Desglose por mutante. Análisis de cada tipo de mutación
 - d. Mutaciones supervivientes. Cuántos mutantes han sido generados y cuántos de ellos han sobrevivido. Justificar por qué han sobrevivido.
4. Justifica qué pruebas son necesarias para “matar” a los mutantes que han sobrevivido e impleméntalas.
5. Re-ejecuta piTest y aporta el nuevo resultado HTML en el que se demuestre que se han eliminado todos los mutantes supervivientes.

Notas:

El código mostrado en el enunciado no está en la solución de la PRAC2, debéis copiar su contenido en vuestro repositorio, dentro del directorio src/test.

Solución

Al revisar el código de pruebas, podemos observar que todas las líneas se ejercitan, lo que da una cobertura del 100%.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <properties>
    <pitest.version>1.15.3</pitest.version>
  </properties>
  <dependencies>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.pitest</groupId>
        <artifactId>pitest-maven</artifactId>
        <version>${pitest.version}</version>
        <dependencies>
          <dependency>
            <groupId>org.pitest</groupId>
```

```

    <artifactId>pitest-junit5-plugin</artifactId>
    <version>1.2.1</version>
  </dependency>
</dependencies>
<configuration>
  <targetClasses>
    <param>edu.uoc.epcsd.course.domain.validator.*</param>
  </targetClasses>
  <targetTests>
    <param>edu.uoc.epcsd.course.domain.validator.*</param>
  </targetTests>
  <outputFormats>
    <param>HTML</param>
    <param>XML</param>
  </outputFormats>
</configuration>
</plugin>
</plugins>
</build>
</project>

```

Ejecutamos pitest:

```
mvn test-compile org.pitest:pitest-maven:mutationCoverage
```

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	100% 14/14	62% 15/24	62% 15/24

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
edu.uoc.epcsd.course.domain.validator	1	100% 14/14	62% 15/24	62% 15/24

Interpretamos las siguientes métricas:

1. **Mutation Coverage.** Se han generado 24 mutantes y 15 han sido eliminados (killed) por las pruebas, por tanto 9 han sobrevivido.

2. **Line Coverage.** Todas las líneas de la clase CourseValidator son ejecutadas. Recordad que una cobertura alta no implica pruebas de calidad, por eso realizamos las pruebas de mutación.

Si navegamos por el package hasta la clase CourseValidator, podemos ver el detalle de los mutantes usados y los que han sobrevivido (en rojo):

```

15 1. negated conditional → KILLED
16 1. replaced boolean return with true for edu/uoc/epcsd/course/domain/validator/CourseValidator::isValidPrice → KILLED
    1. replaced boolean return with true for edu/uoc/epcsd/course/domain/validator/CourseValidator::isValidPrice → SURVIVED
    2. negated conditional → KILLED
18 3. changed conditional boundary → SURVIVED
    4. negated conditional → KILLED
    5. changed conditional boundary → SURVIVED
26 1. negated conditional → KILLED
27 1. replaced boolean return with true for edu/uoc/epcsd/course/domain/validator/CourseValidator::isValidDuration → KILLED
    1. negated conditional → KILLED
    2. replaced boolean return with true for edu/uoc/epcsd/course/domain/validator/CourseValidator::isValidDuration → SURVIVED
29 3. changed conditional boundary → SURVIVED
    4. negated conditional → KILLED
    5. changed conditional boundary → SURVIVED
    1. changed conditional boundary → SURVIVED
39 2. negated conditional → KILLED
    3. negated conditional → KILLED
42 1. negated conditional → KILLED
    2. changed conditional boundary → SURVIVED
43 1. replaced int return with 0 for edu/uoc/epcsd/course/domain/validator/CourseValidator::calculateDiscount → KILLED
44 1. changed conditional boundary → SURVIVED
    2. negated conditional → KILLED
45 1. replaced int return with 0 for edu/uoc/epcsd/course/domain/validator/CourseValidator::calculateDiscount → KILLED
47 1. replaced int return with 0 for edu/uoc/epcsd/course/domain/validator/CourseValidator::calculateDiscount → KILLED

```

Todos los casos están sucediendo porque se han ignorado el uso de valores frontera (boundary).

isValidPrice

#	Mutador	Mutación aplicada	Por qué sobrevive
1	Boundary	price >= MIN_PRICE se muta a price > MIN_PRICE	El test usa 5000L, que sigue siendo > 0. Falta un test con isValidPrice(0L) que espere true.
2	Boundary	price <= MAX_PRICE se muta a price < MAX_PRICE	El test usa 5000L, que sigue siendo < 10000. Falta un test con isValidPrice(10000L) que espere true.
3	Retorno boolean true	La expresión return (price >= 0 && price <= 10000) se muta a return true	El test solo prueba null como caso false. Falta un test con un valor fuera de rango como isValidPrice(10001L) o isValidPrice(-1L) que espere false.

isValidDuration

#	Mutador	Mutación aplicada	Por qué sobrevive
4	Boundary	duration >= MIN_DURATION se muta a duration > MIN_DURATION	El test usa 100L. Falta isValidDuration(1L) que espere true.
5	Boundary	duration <= MAX_DURATION se muta a duration < MAX_DURATION	El test usa 100L. Falta isValidDuration(500L) que espere true.
6	Retorno boolean true	El return de la expresión booleana se muta a return true	El test solo prueba null como caso false. Falta isValidDuration(0L) o isValidDuration(501L) que espere false.

calculateDiscount

#	Mutador	Mutación aplicada	Por qué sobrevive
7	Boundary	price < 0 se muta a price <= 0	El test usa null para cubrir el branch que retorna 0. Falta calculateDiscount(0L) que espere 5 (no 0).
8	Boundary	price >= 5000 se muta a price > 5000	El test usa 7000L. Falta calculateDiscount(5000L) que espere 20.
9	Boundary	price >= 2000 se muta a price > 2000	El test usa 3000L. Falta calculateDiscount(2000L) que espere 10.

La gran mayoría sobreviven por lo mismo, las pruebas usan valores intermedios que no se detectan cuando PIT cambia $\geq a >$ o $\leq a <$. La solución es probar en los límites exactos: el valor mínimo, máximo, y justo fuera de rango.

Código corregido

```
package edu.uoc.epcsd.course.domain.validator;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

class CourseValidatorTest {
```



```
private CourseValidator validator;

@BeforeEach
void setUp() {
    validator = new CourseValidator();
}

@Test
void testIsValidPrice() {
    assertThat(validator.isValidPrice(null)).isFalse();
    assertThat(validator.isValidPrice(-1L)).isFalse();
    assertThat(validator.isValidPrice(10001L)).isFalse();
    assertThat(validator.isValidPrice(0L)).isTrue();
    assertThat(validator.isValidPrice(10000L)).isTrue();
    assertThat(validator.isValidPrice(5000L)).isTrue();
}

@Test
void testIsValidDuration() {
    assertThat(validator.isValidDuration(null)).isFalse();
    assertThat(validator.isValidDuration(0L)).isFalse();
    assertThat(validator.isValidDuration(501L)).isFalse();
    assertThat(validator.isValidDuration(1L)).isTrue();
    assertThat(validator.isValidDuration(500L)).isTrue();
    assertThat(validator.isValidDuration(100L)).isTrue();
}

@Test
void testCalculateDiscount() {
    assertThat(validator.calculateDiscount(null)).isEqualTo(0);
    assertThat(validator.calculateDiscount(-1L)).isEqualTo(0);
    assertThat(validator.calculateDiscount(0L)).isEqualTo(5);
    assertThat(validator.calculateDiscount(1999L)).isEqualTo(5);
    assertThat(validator.calculateDiscount(2000L)).isEqualTo(10);
    assertThat(validator.calculateDiscount(4999L)).isEqualTo(10);
    assertThat(validator.calculateDiscount(5000L)).isEqualTo(20);
    assertThat(validator.calculateDiscount(7000L)).isEqualTo(20);
}
}
```

Si volvemos a ejecutar el informe veremos que se han eliminado todos los mutantes:

Pit Test Coverage Report

Package Summary

edu.uoc.epcsd.course.domain.validator

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	100% 14/14	100% 24/24	100% 24/24

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
CourseValidator.java	100% 14/14	100% 24/24	100% 24/24

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson): Chapter 9. "Testing microservices: Parte 1". Introduction.
- Libro "Microservices Patterns" (Chris Richardson): Chapter 10. "Testing microservices: Parte 2".
- Libro "Fundamentals of Software Architecture" (Richards & Ford).
 - o Chapter 3. "Modularity".
 - o Chapter 6. "Measuring and Governing Architecture Characteristics".

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. Esto incluye la reutilización de soluciones de esta práctica de semestres anteriores. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Los criterios de evaluación de cada ejercicio están indicados en el anexo.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que entregar un solo documento PDF llamado *ApellidosNombre_ISCSDPRAC3.pdf*, donde se explique cómo se ha llevado a cabo la práctica, justificando las decisiones tomadas y proporcionando instrucciones para ejecutar las pruebas.

Además, se debe adjuntar por separado:

1. El código de las pruebas en un archivo ZIP
2. Los resultados HTML en Allure del segundo ejercicio, en otro archivo ZIP independiente: *Ejercicio2.zip*.
3. Los resultados de las pruebas del tercer ejercicio, en otro archivo ZIP independiente: *Ejercicio3.zip*.
4. Los resultados HTML de piTest del cuarto ejercicio, en otro archivo ZIP independiente: *Ejercicio4.zip*.

El documento PDF y los 2 ficheros ZIP se entregarán en el espacio “Entrega de la PRAC3” del aula antes de las **23:59 horas del día 8 de junio de 2026**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación de la Práctica 3

	Excelente (10 puntos)	Notable (7 puntos)	Suficiente (5 puntos)	Insuficiente (2,5 puntos)	Sin respuesta / Todo mal (0 puntos)
Ej. 1	Niveles y tipos de prueba definidos con detalle. Se justifica exhaustivamente cada decisión y el diagrama resumen es coherente con la arquitectura de microservicios	Se definen los niveles correctamente, pero la justificación o la selección de librerías es mejorable. Se incluye el diagrama solicitado.	Se describen los niveles de forma genérica. La justificación se presenta de forma básica y el diagrama carece de detalles o de alguna librería clave	No se identifican correctamente los niveles de prueba o se omite el diagrama resumen y su aplicación al microservicio de Cursos	No se ha contestado o el contenido no tiene relación con estrategias de prueba para microservicios
Ej. 2.1	Se implementan los escenarios de creación y error mediante JUnit5, Mockito y AssertJ. Se justifica técnicamente por qué se requieren mocks y no instanciación directa.	Se implementan los escenarios, pero se omiten detalles en las aserciones o la justificación del uso de mocks se presenta de forma breve.	Se presenta una prueba funcional, pero se hace un uso incorrecto de AssertJ o la justificación sobre dependencias es incompleta.	Se implementa solo un escenario o se utiliza infraestructura real (DB/HTTP), sin usar mocking	No se ha entregado el código o este no compila ni se hace uso de la simulación de dependencias
Ej. 2.2	Se documentan detalladamente los pasos de integración. Se incluyen capturas claras de Overview y Timeline, y se ha entregado el archivo ZIP funcional.	Se realiza la integración correctamente, pero se omiten algunas capturas o la descripción en el PDF se considera poco detallada.	Se genera el informe visual, pero la documentación aportada se muestra escasa. Se entrega el ZIP con errores menores.	No se integra el adaptador de Allure correctamente o no se adjunta el ZIP con los resultados.	No se evidencia la integración con Allure o no se ha entregado el informe HTML en el ZIP.

Ej. 3	Se implementan los dos escenarios en consumidor y proveedor. Se genera el archivo JSON y se aportan evidencias de ejecución de ambos lados.	Se implementan los escenarios, pero las evidencias se muestran confusas o la configuración del PactBuilder es mejorable	Se implementa solo un escenario o se omiten las evidencias de validación en el lado del proveedor.	No se entrega el archivo JSON o el proveedor no valida el contrato. Se detectan errores graves en la lógica.	No se implementan pruebas de contrato o se utilizan mocks simples sin la librería Pact.
Ej. 4	Se analizan las métricas de cobertura y mutantes. Se implementan pruebas adicionales que matan a los mutantes supervivientes, demostrado en el informe y en el ZIP.	Se configura piTest y se identifican mutantes, pero la justificación de su supervivencia no es del todo correcta. Se eliminan la mayoría de mutantes.	Se ejecuta piTest y se entregan informes, pero la implementación de nuevos tests se muestra insuficiente para mejorar la calidad	No se justifican las métricas solicitadas o no se aportan los nuevos resultados tras intentar matar mutantes.	No se entregan los informes de piTest ni se realiza el análisis de la clase CourseValidato r.